

Proyecto 7: Monitoreo de infraestructura con Zabbix

Juan Camilo Ballesteros Sierra
Luis Felipe Murillo Matallana
Juan Sebastián Delgado
Daniela Castro Quiñones

Resumen - Este proyecto implementa una plataforma de monitoreo con Zabbix 6.x, Docker Compose, cuatro servicios monitoreados y alertas validadas con MailHog. La solución se publicó en VPS con HTTPS e incorpora backend Node.js, MariaDB, gráficas, SLO, exporter de métricas, matriz de cumplimiento y pruebas Artillery.

Palabras clave - Zabbix, Docker, monitoreo, alertas, MailHog, Artillery.

I. Introducción

Las infraestructuras telemáticas requieren observabilidad para detectar fallas, analizar disponibilidad y reducir tiempos de indisponibilidad. El objetivo fue construir una solución que evidenciara estado de hosts, recursos, servicios, alertas, históricos y comportamiento bajo carga controlada.

II. Contexto del problema

La red evaluada contiene servicios HTTP, base de datos, DNS y FTP. Sin monitoreo, la detección depende de revisión manual y no queda historial para análisis. La solución debe responder si el host está activo, si el servicio responde, si hay consumo anormal y si la alerta llega por correo.

III. Alternativas de solución

Se compararon Nagios, Prometheus, Datadog y Zabbix. Zabbix fue elegido porque integra agentes, servidor, frontend, base de datos, triggers, dashboards, web escenarios, media types y API de aprovisionamiento en una sola plataforma abierta.

IV. Diseño de la solución

La arquitectura usa Zabbix Server, PostgreSQL, Zabbix Web, MailHog, cuatro servicios monitoreados y agentes Zabbix. El portal público agrega backend Node.js, MariaDB, endpoints JSON, /metrics, /api/charts, /api/live y /api/compliance. En la VPS, Caddy publica subdominios HTTPS.

Herramienta	Fortaleza	Limitación
Nagios	Checks y plugins	Menos integrado para gráficas modernas
Prometheus	Métricas y series de tiempo	Requiere Alertmanager/Grafana
Datadog	Suite gestionada	Servicio externo pago
Zabbix	Agentes, UI, triggers y alertas	Mayor configuración inicial

Host	Servicio	Check
web-host	Node.js web	HTTP 80
db-host	MariaDB	TCP 3306
dns-host	CoreDNS	TCP 53
ftp-host	VSFTPD	FTP 21

ZABBIX

Zabbix docker

Monitoring

Dashboard

Problems

Hosts

Latest data

Maps

Discovery

Services

Inventory

Reports

Configuration

Administration

Support

Integrations

Help

User settings

Sign out

Global view

All dashboards / Global view

System information

Parameter	Value	Details
Zabbix server is running	Yes	zabbix-server:10051
Number of hosts (enabled/disabled)	4	4 / 0
Number of templates	384	
Number of items (enabled/disabled/not supported)	172	172 / 0 / 0
Number of triggers (enabled/disabled [problem/ok])	64	64 / 0 [0 / 64]

Problems

Time	Info	Host	Problem • Severity
------	------	------	--------------------

Fig. 1. Dashboard principal del proyecto en Zabbix.

ZABBIX <<

Zabbix docker

 **Monitoring** ^

Dashboard

Problems

Hosts

Latest data

Maps

Discovery

 Services v

 Inventory v

 Reports v

 Configuration v

 Administration v

 Support

 Integrations

 Help

 User settings v

 Sign out

Hosts

<

Name <input type="text"/>

Host groups <input type="text" value="type here to search"/>

IP <input type="text"/>

DNS <input type="text"/>

Port <input type="text"/>

Severity ☐ Not classified ☐ Warning ☐ High
☐ Information ☐ Average ☐ Disa

Fig. 2. Hosts monitoreados en Zabbix.

ZABBIX

<< < > >>

Zabbix docker

👁 Monitoring

Dashboard

Problems

Hosts

Latest data

Maps

Discovery

🕒 Services

☰ Inventory

📊 Reports

🔧 Configuration

⚙ Administration

🗣 Support

🔗 Integrations

❓ Help

👤 User settings

🔌 Sign out

Latest data

< >

🔍

Host groups

type here to search

Hosts

type here to search

Name

Subfilter affects only filtered data

HOSTS

db-host 43 dns-host 43 ftp-host 43 web-host 43

TAGS

component 168

TAG VALUES

component: application 4 cpu 68 environment 4 memory 28 os 12 sec

DATA

With data Without data

☐ Host	Name ▲
☐ dns-host	Disponibilidad DNS dns-service TCP
☐ ftp-host	Disponibilidad FTP ftp-service
☐ web-host	Disponibilidad HTTP web-service
☐ db-host	Disponibilidad MySQL db-service
☐ web-host	Linux: Available memory ?
☐ db-host	Linux: Available memory ?
☐ dns-host	Linux: Available memory ?
☐ ftp-host	Linux: Available memory ?
☐ web-host	Linux: Available memory in % ?

Fig. 3. Latest data con checks de disponibilidad.

ZABBIX <<

Zabbix docker

Monitoring

Dashboard

Problems

Hosts

Latest data

Maps

Discovery

Services

Inventory

Reports

Configuration

Administration

Support

Integrations

Help

User settings

Sign out

Problems

<

Show

Recent problems

Problems

History

Host groups

type here to search

Hosts

type here to search

Triggers

type here to search

Problem

Severity

☐ Not classified

☐ Warning

☐ High

☐ Information

☐ Average

☐ Disaster

Age less than

☐ 14

days

Time

☐ Severity

Recovery time

Status

Ir

00:06:16

☐ High

PROBLEM

0 selected

Mass update

Fig. 4. Problema activo durante la caída del servicio web.

 MailHog

 GitHub

 Connected

Inbox (21)

 Delete all messages

Jim

Jim is a chaos monkey.
[Find out more at GitHub.](#)

Enable Jim

zabbix@proyecto7.local
 admin@proyecto7.local
Resolved in 15s: ftp-host sin datos del agente
 34 minutes ago
 569 B

zabbix@proyecto7.local
 admin@proyecto7.local
Resolved in 2s: dns-host sin datos del agente
 34 minutes ago
 595 B

zabbix@proyecto7.local
 admin@proyecto7.local
Problem: dns-host sin datos del agente
 34 minutes ago
 578 B

zabbix@proyecto7.local
 admin@proyecto7.local
Problem: ftp-host sin datos del agente
 34 minutes ago
 578 B

zabbix@proyecto7.local
 admin@proyecto7.local
Resolved in 6s: web-host sin datos del agente
 34 minutes ago
 595 B

zabbix@proyecto7.local
 admin@proyecto7.local
Problem: web-host sin datos del agente
 34 minutes ago
 578 B

zabbix@proyecto7.local
 admin@proyecto7.local

Fig. 5. Alertas recibidas en MailHog.

V. Implementación

El despliegue se empaqueta con docker-compose.yml para entorno local y docker-compose.vps.yml para publicación. Zabbix Server usa una imagen personalizada y monta configuraciones como volumen. El script provision_zabbix.py crea grupo, hosts, items, triggers,

dashboard, media type y web scenario mediante API JSON-RPC.

La aplicación web expone endpoints para salud, resumen, base de datos, telemetría, incidentes, carga controlada, gráficas, SLO, cumplimiento y métricas estilo Prometheus. Esto permite observar comportamiento funcional, no solo puertos abiertos.

La configuración final deja trazabilidad entre requisito, archivo y evidencia: Compose define la topología, los Dockerfile y volúmenes prueban personalización, los scripts automatizan Zabbix y las capturas muestran dashboard, hosts, latest data, problemas y correos.

VI. Pruebas

Se validaron dashboard en tiempo real, caída controlada, envío de alertas, métricas históricas y carga con Artillery. En producción, el smoke test generó 96 solicitudes, 0 usuarios fallidos y p95 aproximado de 23.8 ms. La auditoría automática reportó 27 validaciones OK y 0 fallidas.

Durante la sustentación se puede ejecutar una prueba en vivo con artillery-live-demo.yml y detener temporalmente web-service. Con esto el profesor observa tráfico real en el portal, detección de falla en Zabbix, alerta en MailHog y recuperación al reiniciar el servicio.

VII. Discusión de las pruebas

La prueba de caída confirma el flujo completo: falla, problema en Zabbix, alerta en MailHog, recuperación y registro histórico. Las pruebas de carga permiten diferenciar caída total de degradación por tráfico, mientras /metrics y /api/db/status amplían el monitoreo más allá de conectividad básica.

El uso de MailHog evita enviar correos reales durante las pruebas, pero el proyecto conserva una ruta SMTP del dominio para explicar cómo se escalaría la notificación en un ambiente de producción.

VIII. Conclusiones

La solución cumple los requerimientos: despliegue dockerizado, mínimo cuatro hosts monitoreados, dashboards, triggers, alertas y evidencia histórica. El portal público, backend transaccional, SLO, exporter /metrics, Artillery y matriz /api/compliance agregan valor para la sustentación y trazabilidad frente a la rúbrica.

Referencias

[1] Zabbix Documentation. [2] Docker Documentation. [3] MailHog GitHub. [4] Artillery Documentation. [5] Caddy Documentation.